26-10-2026

Actividad 2

Desarrollo Full Stack

Samuel Ramírez Campos
TECMILENIO

Actividad 2: Gestor de tareas con JavaScript

1. Descripción de la aplicación

La aplicación Gestor de Tareas permite agregar actividades, editarlas, marcarlas como completadas, reabrirlas y eliminarlas. Las tareas se muestran dinámicamente en la página. Además, se utiliza LocalStorage para conservar la lista cuando se recarga o se vuelve a abrir la página en el mismo navegador.

2. JavaScript básico y manipulación del DOM

Se utiliza document.querySelector() para seleccionar el formulario, el campo de texto, la lista y los mensajes de la interfaz. addEventListener() detecta las acciones del usuario, como enviar el formulario o pulsar los botones de una tarea. En el evento submit se utiliza event.preventDefault() para evitar que la página se recargue al agregar una tarea.

La función render() actualiza la lista visible después de cada operación. Se crean elementos li, span y div con document.createElement(), y se agregan mediante append(). El método forEach() recorre las tareas para construir la lista. También se usa filter() para contar pendientes y quitar tareas completadas.

3. Características ES6+

Se usa const para referencias que no se reasignan y let no es necesario en la lógica principal porque no hay variables locales que deban cambiar de referencia. Las funciones flecha se utilizan en los manejadores de eventos y en los métodos de arreglo. Los template literals (comillas invertidas) permiten construir clases CSS, mensajes y el HTML de los botones. También se emplean parámetros con valores predeterminados, el operador ?? y la sintaxis moderna de clases.

4. Programación orientada a objetos (POO)

La clase Tarea representa una actividad individual. Su constructor recibe el nombre, el estado (completa o pendiente) y un identificador. El método editar() cambia el nombre de la tarea y actualizarEstado() alterna entre completada y pendiente.

La clase GestorDeTareas mantiene el arreglo de tareas y concentra las operaciones para agregar, editar, cambiar el estado, eliminar, guardar, cargar y mostrar. Cada tarea se instancia con new Tarea(...). Esta separación permite organizar el código y mantener las responsabilidades claras.

5. Validación y accesibilidad

El campo de entrada tiene required y maxlength. Además, JavaScript aplica trim() para rechazar entradas vacías o que solo contengan espacios. Se muestra retroalimentación mediante un elemento con role=status y aria-live. Los botones tienen textos descriptivos y el diseño contempla foco visible para teclado.

6. Almacenamiento con LocalStorage

El método guardar() convierte el arreglo de tareas a una cadena JSON mediante JSON.stringify() y lo guarda con localStorage.setItem(). El método cargar() recupera la cadena con getItem(), la convierte con JSON.parse() y reconstruye instancias de Tarea. Se guarda después de cada cambio para que la información permanezca tras recargar el navegador.

7. Diseño responsivo y pruebas

El archivo styles.css utiliza una cuadrícula flexible de contenido, anchos relativos y una media query para pantallas de hasta 560 píxeles. En móvil, el campo y el botón se apilan y las acciones de cada tarea se acomodan en varias líneas.

Pruebas recomendadas: agregar una tarea válida; intentar agregar un campo vacío; editar una tarea; marcarla como completada y reabrirla; eliminar una tarea; limpiar las completadas; recargar la página para verificar LocalStorage; y usar Chrome DevTools con Toggle device toolbar para revisar una vista de escritorio y otra móvil. Revisar también la consola para confirmar que no existan errores.

8. Archivos del proyecto

- index.html: estructura semántica y enlace a los archivos CSS y JavaScript.
- styles.css: colores, tipografía, botones, lista de tareas y adaptación móvil.
- app.js: clases Tarea y GestorDeTareas, eventos, manipulación del DOM y LocalStorage.

9. Conclusión

La actividad integra HTML, CSS y JavaScript para construir una aplicación interactiva. La clase Tarea modela cada elemento, GestorDeTareas centraliza las operaciones y la manipulación del DOM mantiene la interfaz sincronizada con los datos. ES6+ facilita una sintaxis clara y LocalStorage aporta persistencia en el navegador.

Nota: el almacenamiento se realiza localmente en el navegador y no en un servidor. Para la entrega, se deben agregar las capturas reales de funcionamiento y el enlace al repositorio de GitHub una vez publicado.